

「貪心通常比較簡單，但多半不能解決問題，在演算法與人生中都是這樣。」

孔子說：「及其老也，血氣既衰，戒之在得。」日常生活中最常看到貪的狀況就是政治人物，而想到政治人物就想到金庸的笑傲江湖。這一章的例題習題部分就以笑傲江湖中的人物來串場，不過不必擔心，沒看過故事不影響解題。

這一章介紹貪心演算法 (Greedy algorithm) 以及掃描線演算法 (sweep-line algorithm)，這兩種演算法有點類似，所以放在同一章。這裡的貪心沒有負面的意思，單純指以當前最有利的方式做決定。貪心演算法通常很直覺，所以寫程式往往簡單，但不是對所有問題都能夠找到最佳解。Greedy 通常需要證明，但上機考試與競賽的場合，不需要證明，所以很多人不知道怎麼做的時候就用 Greedy 寫下去猜猜看，考試本來就是該猜就要猜，不過貪心也要貪的正確。掃描線演算法的名字出自計算幾何，是指解題過程想像用一條掃描線沿著某個方向掃描過去，他是一個很好的思考模式，很多經典的分治演算法都有掃描線的版本。有一些數列的問題往往可以看成數線上的問題，所以也可以用掃描線的方式來思考。

貪心演算法與掃描線演算法通常需要搭配排序或優先佇列 (priority_queue，常簡稱 PQ)，我們在第二章介紹過排序了，所以這裡不再重複。優先佇列是一種重要的資料結構，在大部分資料結構的課本中都有介紹自製的方法 (heap) 以及應用，它在 STL 中也有內建的，但它應該不能算“常用簡單”的資料結構，所以跟 set/map 的狀況類似，在 APCS 考試中應該不會出現非用它不可的題目，我們會對它 PQ 做簡單的介紹。

4.1. 基本原理

所謂貪心演算法指的是：一個算法由一連串的決定組成，每次決定時都以當時最有利的方式挑選，而且一旦挑選後就不再更改。我們先以舉一個很簡單的例子來說明。

P-4-1. 少林寺的代幣

令狐冲去少林寺觀光，少林寺有四種代幣，面額分別是 [1, 5, 10, 50]，令狐冲拿了 n 元去換代幣，請問以這四種代幣湊成 n 元，最少要多少枚代幣？

Time limit: 1 秒

輸入格式：第一行是一個正整數 m ，代表有幾筆測資，以下有 m 行，每行是一筆測資，每筆測資是一個正整數 n ， $n < 1000$ 。

輸出：依序每一行輸出每一筆測資的最少的代幣數量。

範例輸入：

3
21
50
123

範例結果：

3
1
7

說明：21 = 10*2 + 1, 50 = 50*1, 123 = 50*2 + 10*2 + 1*3。

因為代幣的數量希望要最少，直覺貪心的想法就是要盡量使用面額大的硬幣，代幣由大面額的往小的考慮，能換盡量換，剩下不足的再考慮比較小的面額。

```
#include <stdio>

int main() {
    int coin[4]={1,5,10,50};
    int m,n;
    scanf("%d",&m);
    for (int i=0;i<m;i++) {
        scanf("%d", &n); // input money
        int num=0; // num of coins
        for (int j=3; j>=0 && n>0; j--) { // large to small
            num += n/coin[j]; // num of coin[j]
            n %= coin[j]; // remaining
        }
        printf("%d\n",num);
    }
    return 0;
}
```

程式寫起來很簡單，簡單到如果考試碰到這種題目，會懷疑是不是有陷阱。這個方法很直覺，但怎麼知道它是對的呢？有些人可能會說：就很直覺啊，路邊隨便找個少林寺掃地的也會這樣做。那麼請看看下面的狀況：假設代幣的面額是[1, 4, 5]三種，上面的貪心方法會找到正確答案嗎？假設 n=8，貪心的方法會先換一個 5 元，然後剩下 3 元就換 3 個 1 元代幣，所以 8 元換了 4 個代幣。但是如果拿兩個 4 元代幣就可以湊 8 元了，所以正確答案應該是 2 而不是 4。請不要懷疑人生，P-4-1 的做法是對的，只是並不是能用在任何場合，我們必須證明。如何證明呢？

在代幣是 $[1, 5, 10, 50]$ 的情形下，如果 1 元的代幣總額不小於 5，一定可以拿出 5 個換成一個 5 元，所以 1 元代幣的個數不會超過 4 個；同樣的你可以證明 5 元代幣不會超過 1 個，10 元代幣不會超過 4 個。也就是說，對任意第 j 種代幣 $\text{coin}[j]$ ，小於 $\text{coin}[j]$ 的代幣總額不會超過 $\text{coin}[j]$ ，所以 Greedy 是對的。在 $[1, 4, 5]$ 的狀況，就沒有這個性質了。順便一提，不要小看少林寺的掃地僧，掃地僧往往是最厲害的。

這個例子中我們看到了貪心法的一些特質。

- 算法由一連串決定組成 (由大到小逐一考慮各種代幣的數量)，每次考慮當前最好決定 (能換盡量換)，選擇後就不再更改。
- 算法的正確性必須經過證明，內容就是要證明所下的決定是對的，也就是一定有一個最佳解包含了貪心法所做的決定，而證明的方法通常都是用換的：假設有一個最佳解跟貪心法做的決定不一樣，那麼，我們可以換成一個與貪心法一樣的決定，而解會更好，或至少不會變差。

在下一節中我們會看到更多的例子。貪心算法往往都是要不斷的找最大值或最小值，如果資料不會變動，可以在開始的時候做一次排序，如果資料是會變動的呢？就必須用一個動態的資料結構來處理，以下我們對優先佇列做一點介紹。

優先佇列 (priority_queue)

第三章介紹過佇列，佇列跟現實生活中排隊的狀況一樣，是先進者先出，但更貼近現實生活中的狀況是有些事情後發生的卻必須先處理，因為緊急或是更重要。優先佇列就是具有優先權的排列，不管進入的順序，優先權高的會排在前面。STL 中有提供內建的 PQ，資料結構的課本中也都有教以 heap 來實作，heap 是一個完全二元樹 (complete binary tree)，但實作在陣列中，也就是心中有樹但程式中沒有樹。heap 的實作並不算困難，但考試與比賽時間寶貴，通常能使用內建的就不要自己寫，因此本書中並不介紹 heap 的實作，有興趣的可以自己去查詢網路或書籍。優先佇列 (PQ) 常用的操作與時間複雜度如下，其中 n 表示成員的佇列的大小：

- 加入新元素 (push)， $O(\log(n))$ 。
- 查詢最大值 (top)， $O(1)$ 。
- 刪除最大值 (pop)， $O(\log(n))$ 。
- 檢查是否為空 (empty)， $O(1)$ 。

請注意刪除與查詢只能針對最大值。以下介紹 PQ 的用法，PQ 內定是最大值在 top，如果是基本型態 (如 int) 而且以內定大小的比較函數，使用起來簡單。它也跟排序一樣可

以使用自定的結構與自定的比較方式，但比較複雜，因為涉及一些物件導向的寫法。我們只介紹一些簡單的用法。以下的範例程式介紹基本用法，包含以 `pair` 來做兩個欄位的資料的寫法，若要改變大小順序，最簡單的方法就是改變 `key` 的正負號。

```
// demo pq
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a[10]={1,4,1,3,8,5,4,9,8,7}, n=10;
    for (int i=0;i<n;i++) printf("%d ",a[i]);
    printf("\nIteratively insert and report max\n");
    priority_queue<int> pq;
    for (int i=0;i<n;i++) {
        pq.push(a[i]);
        printf("%d ",pq.top());
    }
    printf("\n");
    int sum=0;
    while (!pq.empty()) {
        sum+=pq.top();
        pq.pop();
    }
    printf("Delete all and sum=%d\n",sum);
    printf("PQ of struct, top is smallest, insert backward\n");
    priority_queue<pair<int,int>> pq2; // (value, index)
    for (int i=n-1;i>=0;i--) {
        pq2.push({-a[i],i}); // push minus value
        printf("a[%d]=%d, ",pq2.top().second,-pq2.top().first);
    }
    printf("\n");
    return 0;
}
```

在第二章我們介紹過 `set/map`，`set` 也是動態的（可以支援加入與刪除），而且也有大小順序，所以是不是可以用來找 PQ 呢？`set` 的寫法比較簡單用途也比較廣，這樣可以偷懶少學一種。如果我們用 `multiset`（因為可能重複），我們可以在 $O(1)$ 找到最大值，新增與刪除也都是 $O(\log(n))$ 。所以基本上是可以，畢竟會使用 `set` 已經很不錯。下面的範例程式，我們展示以 `set` 來代替並且測試比較兩者的速度。我們把 `set` 的大小關係改成大於，這樣 `begin()` 就是最大值。測試後發現 `set` 的時間超過 PQ 的兩倍以上。所以如果是要參加競賽的人，最好還是多學一種比較好。

```
// compare pq and multiset
#include <bits/stdc++.h>
using namespace std;
#define N 1000000
```

```

multiset<int,greater<int>> S;
priority_queue<int> pq;

int main() {
    int a[N], n=N;
    for (int i=0;i<n;i++) a[i]=rand();

    clock_t t1, t2;
    long long sum=0;
    t1=clock();
    for (int i=0;i<n;i++) {
        S.insert(a[i]);
        sum+=*S.begin();
    }
    while (!S.empty()) {
        sum+=*S.begin();
        S.erase(S.begin());
    }
    printf("%lld\n",sum);
    t2=clock();
    printf("set takes %f sec\n", (float)(t2-t1)/CLOCKS_PER_SEC);
    sum=0;
    t1=clock();
    for (int i=0;i<n;i++) {
        pq.push(a[i]);
        sum+=pq.top();
    }
    while (!pq.empty()) {
        sum+=pq.top();
        pq.pop();
    }
    printf("%lld\n",sum);
    t2=clock();
    printf("PQ takes %f sec\n", (float)(t2-t1)/CLOCKS_PER_SEC);
    return 0;
}

```

4.2. 例題與習題

這一節我們來看貪心與掃描線演算法的題目，我們將題型區分為以下幾類，每一類挑選一些經典題來說明，另外在下一節有一些其他的補充習題：

- 單欄位資料排序
- 結構資料排序
- 以 PQ 處理動態資料
- 貪心搭配二分搜

■ 掃描線演算法

很多貪心算法的重點在證明，但本書以程式實作為主，不以證明為重點，所以有些題目的證明只簡略說明重點，或者請讀者上網路查詢。

4.2.1. 單欄位資料排序

P-4-2. 笑傲江湖之三戰

笑傲江湖的遊戲中，你扮演令狐沖的角色，因為「長恨此身非我有，何時忘卻盈盈」，所以你率領任我行等魔教高手前往少林寺搭救任盈盈，欲知故事原委可參見笑傲江湖第 27 回「三戰」。套句丸尾班長的口頭禪，總而言之，少林寺與五嶽劍派等正教派出 n 位高手，而你方也有 n 位高手，每個人都有一個戰力值。雙方要進行一對一的對戰，每個人不管輸或贏都只能比一場，假設兩人對戰時，戰力值高的獲勝。對於對方的每一位高手，你可以指定派哪一位與其對戰，為了解救盈盈，你希望贏越多場越好，平手則沒有幫助。請計算出最多能贏多少場。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，第二行有 n 個非負整數是對方的戰力，第三行有 n 個非負整數是我方的戰力，同行數字間以空白間格。 n 與戰力值不超過 $1e5$ 。

輸出：輸出最多可能的勝場數。

範例輸入：

```
5
3 1 7 0 2
8 3 5 0 0
```

範例結果：

```
3
```

我們只要關心可以戰勝的那些場次需要誰去出戰，其他場次贏不了，就在剩下的人裡面隨便挑選去應戰。假設我方目前最弱的戰力是 a 而對方最弱的是 b ，如果 $a \leq b$ ，則 a 對誰都不會贏，是沒用的，可以忽略。否則 $a > b$ ，我們宣稱「一定有一個最佳解是派 a 去出戰 b 」，證明如下：

假設在一個最佳解中，a 對戰 x 而 y 對戰 b，我們將其交換改成「a 對 b 而 y 對 x」。根據我們的假設 $y \geq a$ ，交換後 a 可以贏 b 而 y 對 x 的戰績不會比 a 對 x 差，所以交換後勝場數不會變少。

既然如此，我們可以確定派 a 對戰 b 一定可以得到最佳解，並將 a 與 b 移除後，繼續依照上述原則挑選。範例程式如下，因為要不斷的挑選最小值，而戰力不會改變，所以是個靜態的資料。我們可以將雙方戰力先進行一次排序，然後依序由小往大考慮即可。這一題也可以由大到小考慮，基本原則是能贏的贏一點就好，當然可以用優先佇列 (PQ)，在範例程式中有以 PQ 寫的方式，做法簡單，我們就不在此說明，有興趣練習 PQ 的可以自行參考。

```
// P_4_2, one-on-one O(nlogn), sorting+greedy
#include <bits/stdc++.h>
using namespace std;
#define N 100010
int main() {
    int n, enemy[N], ours[N];
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d", &enemy[i]);
    for (int i=0; i<n; i++)
        scanf("%d", &ours[i]);
    sort(enemy, enemy+n); // sort enemy power
    sort(ours, ours+n); // sort our power
    int win=0; // num of win
    for (int i=0, j=0; i<n && j<n; i++) { // for each of our power
        // j is currently weakest enemy
        if (ours[i] > enemy[j]) { // can win some enemy
            win++;
            j++; // next enemy
        }
        // otherwise, ours[i] is useless
    }
    printf("%d\n", win);
    return 0;
}
```

P-4-3. 十年磨一劍 (最少完成時間)

人們常用說「十年磨一劍」來比喻下的功夫深，但是這句話對於華山磨劍坊就不適用了，因為要磨劍的客人非常的多，一把劍如果磨太久，客人等待的時間過長是會被客訴的。華山磨劍坊目前有 n 筆訂單，每筆訂單要磨一把劍，每把劍所需的時間不盡相同。磨劍師傅每次只能磨一把劍，現在希望每一筆訂單的完成時間之總和能夠最小，

希望能找出最好的磨劍順序。舉例來說，如果有四把劍，磨劍需要的時間分別是 (3, 1, 3, 4)，如果以 (3, 1, 3, 4) 的順序來磨，第一把的完成時間是 3，第二把完成時間是 3+1=4，第三把是 3+1+3=7，第四把是 3+1+3+4=11，總和是 3+4+7+11=25。如果把順序改成 (1, 3, 3, 4)，那麼完成時間分別是 (1, 4, 7, 11)，總和是 23，這是這一題最好的解。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，第二行有 n 個正整數是每一把劍需要的時間，同行數字間以空白間格。 n 與每把劍的時間都不超過 $1e5$ 。

輸出：輸出最小的完成時間總和。

範例輸入：

```
4
3 1 3 4
```

範例結果：

```
23
```

有 n 個工作要安排一個順序後依序完成，希望每個工作的完成時間總和最小，這是個經典的 minimum completion time 問題。直覺上短的工作要先做，放在越前面的工作，等它的人越多。如果把計算時間的方式仔細看一下，你會發現如果 $job[i]$ 是第 i 個工作所需的時間，那麼完成總時間是 $\sum_{i=0}^{n-1} (n-i) \times job[i]$ ，所以 shortest-job-first 是對的。另外一個證明方法是假設 $job[i]$ 的順序是最佳解而存在某個工作 $job[i] > job[i+1]$ ，那麼，我們交換這兩個相鄰工作，其他的工作時間不會改變，但這兩個工作的完成時間總和會減少，如此可知，最佳解中不可能大的排在小的前面。這種考慮相鄰交換的情形是個證明的技巧，因為其它的工作不會改變，可以得到很簡單的證明。

程式非常簡單，排序後計算就好了，但是不要把計算寫成 $O(n^2)$ 的笨方法了，運用 prefix-sum 的手法就很容易在排序後以 $O(n)$ 完成計算，另外一個方法是用上面的公式。

```
// P_4_3, min waiting time, sorting+greedy
#include <bits/stdc++.h>
using namespace std;
#define N 100010
int main() {
    int n, job[N];
```



```

scanf("%d",&n);
for (int i=0;i<n;i++)
    scanf("%d",&job[i]);
sort(job, job+n); // sort from small to large
long long int comp=0, total=0;
for (int i=0; i<n; i++) {
    comp += job[i]; // completion time of i
    total += comp; // total completion time
}
printf("%lld\n",total);
return 0;
}

```

4.2.2. 結構資料排序

在上面兩個例題中，每個資料都只是一個數字，接下來我們要看需要使用 struct 來存資料的狀況。在第二章我們介紹過結構資料的排序，自訂結構排序時，必須寫一個比較函數，當然也可以利用 STL 中 pair 來偷懶，如果不熟悉的人，請再次參考第二章的方法。下面這一題是很有名的經典題(activity selection problem)。

P-4-4. 幾場華山論劍(activity selection)

自從華山論劍聲名大噪之後，想參加的人絡繹不絕，因此華山派決定每年都舉辦非常多場的華山論劍，每一場都有自己的開始時間與結束時間，參加者必須全程在場，所以不能在同一時間參加兩場。令狐冲拿到了今年所有場次的資料，希望能參加越多場越好，以便盡速累積經驗值，請你幫忙計算最多可以參加幾場。請注意，所挑選活動必須完全不重疊，兩場活動只在端點重疊也是不可以同時參加的，也就是說前一場的結束時間必須小於下一場的開始時間。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，代表一共舉辦了 n 場華山論劍，接下來 n 行，每行兩個非負整數 s 與 t ，代表這場活動的時間區間是 $[s, t]$ ，兩數字間以空白間格。 n 不超過 $1e5$ ， $s < t < 1e9$ 。

輸出：最多參加幾場活動。

範例輸入：

```

4
1 4
0 3
3 4

```

4 6

範例結果：

2

說明：可以挑選 $[0, 3]$ 與 $[4, 6]$ 兩場活動。

這個題目簡單說就是：數線上有若干線段，要挑選出最多的不重疊的線段。第一個想法可能是盡量挑短的，但是範例中有一個反例，挑短的不是正確答案。我們宣稱下面這個的性質：

「若 $[x, y]$ 是所有現存線段中右端點最小的，那麼一定有個最佳解是挑選了 $[x, y]$ 。」

證明：假設最佳解沒有挑 $[x, y]$ ，令 $[a, b]$ 是最佳解中右端點最小的。根據我們的假設， $y \leq b$ ，因此將 $[x, y]$ 取代 $[a, b]$ 不會重疊到任何最佳解中的其他線段，我們可以得到另外一個最佳解包含 $[x, y]$ 。

根據此性質，我們可以不斷地將最小右端的線段挑選進解答，然後略過任何與它重疊的線段。因為線段不會改變，一開始依照右端由小排到大，然後從頭到尾掃描一次就可以得到答案。以下是範例程式，在此程式中我們定義了一個 struct 包含兩個整數欄位，用來存放一個線段的左右端點。為了排序，定義了一個比較函數 cmp，用來比較線段的右端點，對結構排序不熟悉的請複習第二章。在排序後，以一個迴圈從頭到尾掃描一次，以變數 endtime 保持著前一個已挑選活動的結束時間（初值給-1 因為輸入的時間皆為非負），迴圈內以 if 來判斷 ac[i] 開始時間是否大於 endtime，若是，挑選起來，修改答案與 endtime；若否，可以直接丟棄它（不做任何事）。

```
// P_4_4, activity selection, sorting+greedy
#include <bits/stdc++.h>
using namespace std;
#define N 100010
struct ACT{
    int s, f; // start and finish time
};
// compare finish time
bool cmp(ACT p, ACT q) {
    return p.f < q.f;
}
int main() {
    int n;
    ACT ac[N];
    scanf("%d", &n);
    for (int i=0; i<n; i++)
        scanf("%d%d", &ac[i].s, &ac[i].f);
    sort(ac, ac+n, cmp); // sort from small to large
    int endtime=-1, total=0;
```

```

for (int i=0; i<n; i++) {
    if (ac[i].s>endtime) { // compatible
        total++;
        endtime=ac[i].f;
    }
}
printf("%d\n",total);
return 0;
}

```

我們當然可以利用 STL 的 pair 來逃避寫比較函數，以下是這樣的寫法，有興趣的參考。我們用 vector 來寫，你當然可以用陣列來寫，重點是把右端點放在第一個欄位，因為 pair 的大小關係是依欄位順序的字典順序。

```

// P_4_4b, activity selection, sorting+greedy
#include <bits/stdc++.h>
using namespace std;
#define N 100010

int main() {
    int n;
    vector<pair<int,int>> act;
    scanf("%d",&n);
    for (int i=0;i<n;i++) {
        int s,t;
        scanf("%d%d",&s, &t);
        act.push_back({t,s}); // key = finish time
    }
    sort(act.begin(), act.end()); // sort by finish
    int endtime=-1, total=0; // end of previous activity
    for (auto p: act) {
        if (p.second>endtime) { // compatible
            total++;
            endtime=p.first; // set end-time
        }
    }
    printf("%d\n",total);
    return 0;
}

```

在 P-4-3 十年磨一劍的例題中，每一件磨劍工作，無論需要時間的長短，重要性都是一樣的，我們要求的只是完成時間的總和最小化，下面是一個變化題，每一個工作延後的代價不同。

P-4-5. 嵩山磨劍坊的問題 (加權最小完成時間)

嵩山磨劍坊接了 n 筆磨劍工作，磨劍師傅每次只能磨一把劍。除了每把劍各有所需的時間之外，每件工作的重要性也不同。假設第 i 筆訂單需要的時間是 $t[i]$ ，而重要性是 $w[i]$ 。磨劍坊的計價方式是：每件工作都已經先收了一筆款項，假設第 i 筆訂單在時間 f 時完成，則需要扣款 $f*w[i]$ ，現在希望將 n 筆磨劍工作安排最好的順序，使得扣款總額越小越好。嵩山派掌門左冷禪是非常嚴厲的老闆，希望你能幫磨劍師傅找出最好的順序以免他遭到處罰。

舉例來說，如果有四把劍，磨劍需要的時間分別是 $t=(1, 4, 5, 6)$ ，而重要性依序是 $w=(1, 3, 4, 7)$ 。如果依訂單編號順序 $(1, 2, 3, 4)$ 來磨，也剛好是工作時間由短到長的順序，每件工作的完成時間依序是 $(1, 5, 10, 16)$ ，扣款總額是 $1*1 + 5*3 + 10*4 + 16*7 = 168$ 。如果依照訂單編號順序 $(4, 1, 3, 2)$ 來磨，則 t 與 w 重新排列後分別是 $t=(6, 1, 5, 4)$ ， $w=(7, 1, 4, 3)$ 。完工時間是 $(6, 7, 12, 16)$ ，扣款總額是 $6*7 + 7*1 + 12*4 + 16*3 = 145$ 。這是這一題 24 種排列中最好的解。

Time limit: 1 秒

輸入格式：輸入的第一行工作數 N ， $N \leq 1e5$ 。第二行有 N 個正整數，依序是各訂單所需時間 $t[1]$ 、 $t[2]$ 、...、 $t[N]$ 。第三行有 N 個非負整數，依序是各訂單的重要性 $w[1]$ 、 $w[2]$ 、...、 $w[N]$ ，時間與重要性皆不超過 1000，相鄰以空白間隔。

輸出：輸出最小的扣款總額。答案不超過 $1e18$ 。

範例輸入：

```
4
1 4 5 6
1 3 4 7
```

範例結果：

```
145
```

如果每個工作(訂單)的重要性(w)都是一樣的話，這一題跟 P-4-3 就是一樣的，答案就是依照工作時間以 Shortest-job first 的方式排列，但是在有權重的狀況下，題目範例中就告訴你時間短的不一定在前面。我們再想想看，只看時間的話，時間短的應該排前面；那麼如果只看權重的話呢？權重大的應該排前面，因為延遲的話罰得重。但是兩個都要考慮的話，就必須兩個因子都列入考慮。我們宣稱以下特性：最佳解中，排在前面的， t/w 值一定比較小。證明？跟 P-4-3 類似，考慮排在相鄰的兩個工作，假設 $t[i]/w[i] > t[i+1]/w[i+1]$ 。我們交換兩工作，其他的工作完成時間不會改變，而 i 延後了 $t[i+1]$ 的時間， $i+1$ 則提早了 $t[i]$ 的時間，因此變化量為 $w[i]*t[i+1] - w[i+1]*t[i] < 0$ ，也就是解會變好。程式的寫法不難，但是需

要兩個欄位 struct 的排序，而大小的比較函數是以交叉相乘的方式。這裡要提醒，在類似的情形下要注意資料範圍與 overflow 的問題。以這一題的範圍，1~1000，相乘之後不會有問題，如果數字比較大，相乘之前可能需要轉 long long，另外分數的比較也要注意負號，如果有分母有可能是負整數，交叉相乘要小心。

```
// P_4_5 greedy, using c++ sort+struct
#include <bits/stdc++.h>
using namespace std;
struct Item {
    int t,w;
};
bool comp(Item p, Item q) {
    return p.t * q.w < p.w * q.t;
}

int main() {
    int n;
    scanf("%d", &n); assert(n<100000);
    Item Q[N];
    for (int i=0; i<n; i++)
        scanf("%d", &Q[i].t);
    for (int i=0; i<n; i++)
        scanf("%d", &Q[i].w);
    for (int i=0; i<n; i++)
        assert(Q[i].t>0 && Q[i].t<=1000 \
            && Q[i].w>=0 && Q[i].w<=1000);
    sort(Q, Q+n, comp);
    long long ans=0, comp_t=0; // completion time
    for (int i=0; i<n; i++) {
        comp_t += Q[i].t;
        ans += comp_t * Q[i].w;
    }
    printf("%lld\n", ans);
    return 0;
}
```

或許有人想知道，既然是依照 t/w 排序，何不直接相除當做 key？是可以，但必須考慮兩個問題，第一是精準度夠不夠，是否會發生捨位誤差造成順序錯誤。以這一題來說是不會。第二個問題，分母是否有可能是 0？這一題是會。所以需要特殊處理一下，下面的範例程式是以這個方式寫的，另外用了 STL 的 tuple，因為有三個欄位。這個寫法其實沒有比較簡單，其實並不推薦，有興趣的參考就好。

```
// P_4_5b greedy, using c++ sort+tuple
// key of type double t/w, oo for weight 0,
#include <bits/stdc++.h>
using namespace std;
```

```

#define N 100010
#define oo 10000.0
int main() {
    int n;
    scanf("%d", &n);
    vector<tuple<double,int,int>> Q;
    for (int i=0; i<n; i++) {
        int t;
        scanf("%d", &t);
        Q.push_back({0,t,0});
    }
    long long total_w=0; // total weight
    for (int i=0; i<n; i++) {
        int w;
        scanf("%d", &w);
        total_w+=w;
        get<2>(Q[i])=w;
        get<0>(Q[i])=(w==0)? oo: (double)get<1>(Q[i])/w;
    }
    sort(Q.begin(), Q.end());
    long long ans=0; // another way to compute
    for (int i=0; i<n; i++) {
        ans += total_w * get<1>(Q[i]);
        total_w -= get<2>(Q[i]);
    }
    printf("%lld\n", ans);
    return 0;
}

```

下面一題很類似，因此留做習題，不多解釋。

Q-4-6. 少林寺的自動寄物櫃 (APCS201710)

少林寺的自動寄物櫃系統存放物品時，是將 N 個物品堆在一個垂直的貨架上，每個物品各佔一層。系統運作的方式如下：每次只會取用一個物品，取用時必須先將在其上方的物品貨架升高，取用後必須將該物品放回，然後將剛才升起的貨架降回原始位置，之後才會進行下一個物品的取用。

每一次升高貨架所需要消耗的能量是以這些物品的總重來計算。現在有 N 個物品，第 i 個物品的重量是 $w(i)$ 而需要取用的次數為 $f(i)$ ，我們需要決定如何擺放這些物品的順序來讓消耗的能量越小越好。

舉例來說，有兩個物品 $w(1)=1$ 、 $w(2)=2$ 、 $f(1)=3$ 、 $f(2)=4$ ，也就是說物品 1 的重量是 1 需取用 3 次，物品 2 的重量是 2 需取用 4 次。我們有兩個可能的擺放順序（由上而下）：

- $(1, 2)$ ，也就是物品 1 放在上方，2 在下方。那麼，取用 1 的時候不需要能量，而每次取用 2 的能量消耗是 $w(1)=1$ ，因為 2 需取用 $f(2)=4$ 次，所以消耗能量數為 $w(1) * f(2)=4$ 。
- $(2, 1)$ 。取用 2 的時候不需要能量，而每次取用 1 的能量消耗是 $w(2)=2$ ，因為 1 需取用 $f(1)=3$ 次，所以消耗能量數= $w(2) * f(1)=6$ 。

在所有可能的兩種擺放順序中，最少的能量是 4，所以答案是 4。再舉一例，若有三物品而 $w(1)=3$ 、 $w(2)=4$ 、 $w(3)=5$ 、 $f(1)=1$ 、 $f(2)=2$ 、 $f(3)=3$ 。假設由上而下以 $(3, 2, 1)$ 的順序，此時能量計算方式如下：取用物品 3 不需要能量，取用物品 2 消耗 $w(3) * f(2)=10$ ，取用物品 1 消耗 $(w(3)+w(2)) * f(1)=9$ ，總計能量為 19。如果以 $(1, 2, 3)$ 的順序，則消耗能量為 $3*2+(3+4)*3=27$ 。事實上，我們一共有 $3!=6$ 種可能的擺放順序，其中順序 $(3, 2, 1)$ 可以得到最小消耗能量 19。

Time limit: 1 秒

輸入格式：輸入的第一行是物品件數 N ， $N < 1e5$ 。第二行有 N 個正整數，依序是各物品的重量 $w(1)$ 、 $w(2)$ 、...、 $w(N)$ 。第三行有 N 個正整數，依序是各物品的取用次數 $f(1)$ 、 $f(2)$ 、...、 $f(N)$ ，重量與次數皆不超過 1000，相鄰以空白間隔。

輸出：輸出最小能量消耗值。答案不超過 $1e18$ 。

範例輸入：

```
3
3 4 5
1 2 3
```

範例結果：

```
19
```

4.2.3. 以 PQ 處理動態資料(*)

這一小節中我們提出需使用優先佇列 (priority queue, PQ) 的題目。由於 APCS 的範圍應該不包含 PQ，所以照理說不太會出現在 APCS 考試中。

P-4-7. 岳不群的併派問題 (Two-way merge) (*)

江湖上門派林立，華山派掌門岳不群認為要消除門派的衝突就是要合併各門派，但不能操之過急，必須每次挑兩個門派合併，如此逐步合併，最後將所有門派合併成一派。合併門派需要一些成本，每個門派都有他的成員數，第 i 個門派的成員數是 $m(i)$ ，合併兩派的成本就是雙方成員數之和，而兩派合併後，雙方成員就會歸到新合併的門派。岳不群不希望耗費太多成本，輸入各派人數，請幫他計算最少的合併總成本。

舉例來說，一開始如果有三派， $m=(3, 5, 1)$ ，若 3 人與 5 人的兩派先合併，成本是 $3+5=8$ ，合併後剩下兩派，人數是 $(8, 1)$ ，再合併這兩派，成本是 $8+1=9$ ，此時已經只剩下一派，也就是完成所有合併統一江湖了，總成本是 $8+9=17$ 。如果我們更換合併的順序，先合併 3 人與 1 人的兩派 (成本 4)，再合併剩下的 5 人 (成本 $4+5=9$)，則總成本只有 $4+9=13$ 。這是這個例子的最低成本。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，代表初始的門派數量，第二行有 n 個正整數是每派的成員數，同行數字間以空白間格。 n 不超過 $2e5$ ，每個門派初始人數都不超過 $1e5$ 。

輸出：第一行輸出總人數，第二行輸出最小的合併總成本。

範例輸入：

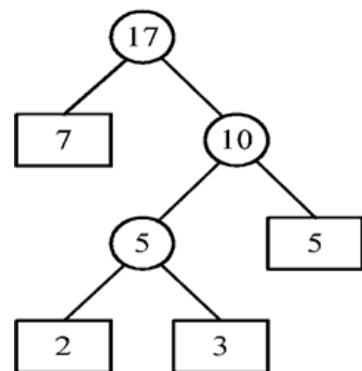
```
3
3 5 1
```

範例結果：

```
9
13
```

這一題事實上是 Two-way merge tree 的問題，與一個有名的 Huffman code 是等價的。正確的合併順序的計算方法是：每次挑選人數最少的兩派合併，重複此步驟直到剩下一派為止。任何一種合併的過程可以用一個二元樹表示，右圖是一個例子，圖中每個方塊表示一個初始的門派，圓圈表示一個合併。初始人數 $m(i)$ 是 $(7, 2, 3, 5)$ ，先合併 $(2, 3)$ 變成 5，再合併 $(5, 5)$ 變成 10，最後合併 $(7, 10)$ 變成 17。合併的成本圓圈中的數字相加，這個例子 $5+10+17=32$ 就是最低成本。這一

題的證明稍嫌麻煩，我們在此不給完整的證明，只做一些說明。總成本也可用另外一個方式計算，每一個方塊往上走到頂會經過幾次圓圈 (稱為深度)，就代表他經過幾次合併，所以每個 $m(i)$ 乘上他的深度加總後就是總成本，所以直覺上看得出來，數字越大



的要放在深度小的地方，也就是說數字越小的要越早合併，完整的證明留給讀者上網搜尋 Huffman code 或 two-way merge tree。

雖然證明有點麻煩，但是程式非常好寫，我們只要每次找出最小的兩個數，將他們刪除，再將他們的和放入。重複這個步驟直到剩下一個數字。但是要有效率的做到必須借助資料結構，如果使用排序，每次刪除最小值或許不是大問題（記住有效位置，每次後移兩格就好），但是插入兩數相加的和就會很浪費時間了。以下是用陣列的方法寫的範例程式，時間複雜度是 $O(n^2)$ 。雖然他效率不夠好，但是我們也使用了一些技巧來減少資料搬移的動作，所以也算個好的練習。

```
// P_4_7 greedy, slow method using array
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
#define N 400010
int main() {
    int n;
    LL m[N];
    scanf("%d", &n);
    for (int i=0; i<n; i++) {
        scanf("%lld", &m[i]);
    }
    sort(m,m+n);
    LL left=0, right=n-1, cost=0; // [left, right]=current data
    // merge, n reduced by 1 each time
    for ( ; n>1; n--) {
        // merge the smallest two sets
        LL x=m[left]+m[left+1];
        cost += x;
        // delete 0 and 1, insert x
        int j=right;
        left += 2; // delete two smallest
        while (j>=left && m[j]>x) { // find position
            m[j+1]=m[j];
            j--;
        }
        m[j+1]=x; // insert x
        right++;
    }
    printf("%lld\n%lld\n", m[left], cost);
    return 0;
}
```

要能夠有效率的做到增刪資料，我們必須借助資料結構，優先佇列每次可以找到最大值（或最小），是這一題最適合的資料結構，只要每次 pop 出兩個資料，相加後 push 回去就可以了。注意數值的大小以及我們放在 PQ 中的要變號，以便每次找到最小值。檔案中還有一支程式是以 multiset 取代 PQ，留給有興趣的人參考，這裡就不多說明了。

```

// P_4_7 greedy, using PQ
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

int main() {
    int n;
    scanf("%d", &n);
    priority_queue<LL> PQ;
    for (int i=0; i<n; i++) {
        int m;
        scanf("%d", &m);
        PQ.push(-(LL)m); // change sign (to find minimum)
    }
    LL cost=0; // total cost
    for (int i=0; i<n-1; i++) { // merge n-1 times
        // merge the smallest two sets
        LL m=PQ.top();
        PQ.pop();
        m += PQ.top();
        PQ.pop();
        PQ.push(m);
        cost += m;
    }
    printf("%lld\n%lld\n", -PQ.top(), -cost);
    return 0;
}

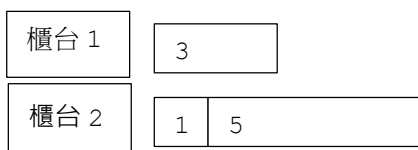
```

下面一個題目是在工作排程時的問題，做法簡單，但證明需要想一下，留做習題，如果資料量不大，可以用陣列做，但資料量大時，必須使用資料結構。

Q-4-8. 先到先服務 (*)

有 n 個客人要分配到 m 個櫃台來服務，客人依編號順序到達，第 i 個客人需要 $t(i)$ 的時間來完成（無論分配到哪一個櫃台），而且每個客人的需求必須在同一個櫃台完成。因為公平的因素，先到客人不可以比晚到客人更晚開始服務。現在希望安排每個客人的服務櫃檯，以便可以在最短的時間內完成所有客人的需求。

舉例來說，如果有三位客人與兩個櫃台，需求的時間依序是 $(3, 1, 5)$ ，我們可以如下圖分配，最後的完成時間是 6。



請注意，我們不可以如下分配，雖然這樣的完成時間會減少到 5，但是第 3 個客人比第 2 個客人早開始服務違反了先到先服務的原則。

Time limit: 1 秒

輸入格式：輸入兩行，第一行是正整數 n 與 m ，第二行是 n 個正整數 $t(1)$,

櫃台 1	3	1
櫃台 2	5	

$t(2), \dots, t(n)$ ，同行數字間以空白間格。 n 與 m 不超過 $2e5$ ， $t(i)$ 不超過 $1e4$ 。

輸出：最早可能服務完所有客人的時間。

範例輸入：

```
3 2
3 1 5
```

範例結果：

```
6
```

4.2.4. 外掛二分搜

貪心演算法經常可以搭配二分搜，以下舉兩個例子。

P-4-9. 基地台 (APCS201703)

直線上有 N 個要服務的點，每架設一座基地台可以涵蓋直徑 R 範圍以內的服務點。輸入服務點的座標位置以及一個正整數 K ，請問：在架設 K 座基地台以及每個基地台的直徑皆相同的條件下，基地台最小直徑 R 為多少？

Time limit: 1 秒

輸入格式：輸入有兩行。第一行是兩個正整數 N 與 K ，以一個空白間格。第二行 N 個非負整數 $P[0], P[1], \dots, P[N-1]$ 表示服務點的點座標，相鄰數字以空白間隔。座標範圍不超過 $1e9$ ， $1 \leq K < N \leq 5e4$ 。

輸出：最小的基地台直徑。

範例輸入：

6 2
5 2 1 7 5 8

範例結果：

3

服務點可以看成數線上的點，基地台可以看成數線上的線段，要計算以 K 根長度相同的線段蓋住所有的點，最小的線段長度。這一題是給數量 K 要求長度 R 。要解決這個問題，先看以下問題：輸入給數線上 N 個點以及 R ，請問最少要用幾根長度 R 的線段才能蓋住所有輸入點。

我們以貪心法則來思考。考慮座標值最小的點 P ，如果有一個解的最左端點小於 P ，我們可以將此線段右移到左端點對齊 P ，這樣的移動不會影響解的合法性，因此，可以得到以下結論：「一定有一個最佳解 (最少的 K) 是將一根線段的左端放在最小座標點上。」因此，我們可以放上一個線段在 $[P, P+R]$ ，然後略過所有以被蓋住的點，對剩下的點繼續此步驟。重複執行到所有的點被蓋住，就可以找到最小的 K 值。

回到我們的原來問題。假設 $f(R)$ 是給定長度 R 的最少線段數，那麼上述的演算法可以求得 $f(R)$ 。很直覺地，當 R 增加時， $f(R)$ 必然只會相同或減少，因為用更長的線段去蓋相同的點，不會需要更多線段。所以我們可以二分搜來找出最小的 R ，滿足 $f(R) \leq K$ 。

以下是範例程式。先寫一個函數 `enough(x)` 計算長度 x 的線段是否足夠，我們不需要算出最少的線段數量，只要知道 K 根是否足夠即可。主程式中將點座標排序後，就以二分搜來找出最長的“不足夠”長度，最後加一就是最短的足夠長度。這裡的二分搜採取第二章介紹的一路往前跳的方式，範例程式中也有傳統維護左右範圍的二分搜寫法，這裡就不多做說明。

```
// P_4_9 min length R to cover points by k segment of same length
// O(nlogL), L is distance
#include <bits/stdc++.h>
using namespace std;
#define N 50010
int p[N], n, k;
// check if k segment of length r is enough
bool enough(int r) {
    int nseg=k, endl = -1; // current covered range
    for (int i=0; i<n; i++) {
        if (p[i] <= endl) continue;
        if (nseg == 0) return false;
        // use a segment to cover
```

```

        nseg--; // remaining segments
        endlene = p[i] + r;
    }
    return true;
}

int main() {
    scanf("%d%d", &n, &k);
    for (int i=0; i<n; i++)
        scanf("%d", p+i);
    sort(p, p+n);
    // binary search, jump to max not-enough length
    int len = 0, L = p[n-1] - p[0];
    for (int jump=L/2; jump>0; jump>>=1) {
        while (len+jump<L && !enough(len+jump))
            len += jump;
    }
    printf("%d\n", len+1);
    return 0;
}

```

下一題是類似的習題。

Q-4-10. 恢復能量的白雲熊膽丸

令狐沖闖黑木崖要依序通過 n 個關卡，第 i 個關卡要消耗 $p[i]$ 的能量，如果能量不足就會闖關失敗。不管當時的能量剩下多少，吃下一顆恆山派的「白雲熊膽丸」就會讓令狐沖的能量值恢復到滿額的 F 。假設在開始時能量是 F ，闖關過程中最多只能服用 m 次，輸入 $p[]$ 與 m ，請問令狐沖的能量額 F 至少必須是多少才能闖關成功。請注意，連吃兩顆是沒用的，能量還是 F 。

Time limit: 1 秒

輸入格式：第一行是正整數 n 與非負整數 m ，第二行，有 n 個正整數 $p[1], p[2], \dots, p[n]$ 。 $p[i]$ 不超過 $1e5$ ， $0 \leq m < n \leq 1e5$ 。

輸出：輸出 F 的最小值。

範例輸入：

```

7 2
3 4 1 7 4 1 2

```

範例結果：

```

8

```

說明：闖過三關(3, 4, 1)，吃藥，闖過(7)，再吃藥，闖過最後 3 關(4, 1, 2)。F=7 的話至少要吃三次藥才能通過。

4.2.5. 掃描線演算法 sweep-line

掃描線演算法的名字來自在解某些幾何問題時，想像用一條掃描線沿著某個方向掃描，過程中記錄某些資料或維護某些結構。因此與貪心演算法類似，往往第一步就是排序。常見的題目有數線上的點或線段，以及平面上的點或直線。限於難度，這裡僅提出一些經典而不太複雜的題目。

P-4-11. 線段聯集 (APCS 201603)

輸入數線上的 N 個線段，計算線段聯集的總長度。

Time limit: 1 秒

輸入格式：第一行是一個正整數 N ，接著的 N 行每一行兩個非負整數，代表一根線段的左端點與右端點，左端點座標值小於等於結束端點座標值，兩者之間以一個空格區隔。 N 不超過 $1e5$ ，座標絕對值皆不超過 $1e8$

輸出：線段聯集總長度。

範例輸入：

```
5
10 20
20 20
30 75
5 15
40 80
```

範例結果：

```
65
```

這一題有個比較簡單的常見版本是座標範圍很小而且線段數量也不多，例如 N 是 1000 而座標範圍在 $0 \sim 1000$ 。在此情形下把 $[0, 1000]$ 這個區間看成 1000 個長度為 1 的區段，以一個陣列 `seg[]` 來記錄每一個區段是否有被線段覆蓋。具體來說：

`seg[i] = 1` if 區間 $[i, i+1]$ 有被覆蓋，否則 `seg[i] = 0`。

初始時，所有 $\text{seg}[i] = 0$ ，表示沒有線段。對於每一個輸入的線段 $[\text{left}, \text{right}]$ ，我們將陣列 $\text{seg}[\text{left}] \sim \text{seg}[\text{right}-1]$ 的區段都設為 1，模擬將線覆蓋在區間上。等到所有線段都處理完畢後，數一數有多少區段被設為 1 就知道覆蓋的總長度了。

```
// P-4-11, subtask, table method
#include <stdio.h>

int main() {
    int i, left, right, n;
    int seg[1001]={0}, total=0;
    scanf("%d", &n);
    for (i=0; i<n; i++) {
        scanf("%d%d", &left, &right);
        for (j=left; j<right; j++)
            seg[j] = 1;
    }
    for (i=0; i<1000; i++)
        total += seg[i];
    printf("%d\n", total);
    return 0;
}
```

這個方法的複雜度是線段長度的總和，在大座標範圍時的效率很差。若要與座標範圍無關，基本想法是將線段逐一加入，維護好目前的聯集 S ， S 顯然是一群不相連的線段。每次加入一根線段時，如果他與 S 中的線段沒有交集，那太好了，把他放進去就行了；如果有交集呢？就必須將有交集的部分找出來做成聯集。這樣做不是不行，但是非常的麻煩。我們試試看掃描線的想法。

想像有根掃描線，從左往右掃，過程中我們一樣保持住目前已經看到的線段的聯集 S 。掃描過程可能碰某個線段的左端或是右端，其他的位置對線段聯集的變化並無影響可以忽略。碰到某線段左端時，代表有一個線段要加入 S 。從左往右掃的好處是，當一個線段 $[x, y]$ 加入時，最多只會跟 S 中的一根線段(最後一根)有交集！因為他的左端 x 不會在 S 中任何一根線段的左方(記得吧，我們從左往右)。什麼時候沒交集呢？如果 x 大於 S 中的最大右端。而且，如果這個線段與 S 沒交集，後面的線段也都不會再跟 S 有交集，因為後面的線段的左端都大於等於 x 。這麼一來，程式就很好寫了，我們只要依照線段左端從小到大的順序逐一將線段加入 S ，維護好 S 中的最後一根線段就好了。以下是範例程式，我們定義了一個 `struct` 結構來放線段，並且定義一個比較函數 `cmp()` 以便依照線段左端排序，時間複雜度 $O(n \log(n))$ ，因為排序。

```
//P4-11 union of segments, using struct
#include <bits/stdc++.h>
using namespace std;
```

```

#define N 100010
struct Seg {
    int left, right;
};
bool cmp(Seg p, Seg q) {
    return p.left < q.left;
}
int main() {
    int n, total=0; // total length
    Seg s[N];
    scanf("%d",&n);
    for (int i=0; i<n; i++)
        scanf("%d%d", &s[i].left, &s[i].right);
    sort(s, s+n, cmp); // sort by left
    Seg last = s[0]; // last segment in the union
    for (int i=1; i<n; i++) { // insert each segment
        if (s[i].left > last.right) { // disjoint
            total += last.right - last.left;
            last = s[i];
            continue;
        }
        // else part, merge last and s[i]
        last.right = max(last.right, s[i].right);
    }
    total += last.right - last.left; // don't forget
    printf("%d\n",total);
    return 0;
}

```

偷懶的方法就是利用 pair 來放線段，把排序的依據放在 first，可以省卻寫比較函數。其實 C++ 的 struct 與比較函數都很好寫 (比起 C)，也不一定要學偷懶的方法。下面還是列出這個寫法給有興趣的人參考。

```

//P4-11 union of segments, using pair
#include <bits/stdc++.h>
using namespace std;
#define Left first
#define Right second

int main() {
    int i,n,total;
    vector<pair<int,int>> s;
    scanf("%d",&n);
    for (i=0; i<n; i++) {
        int p, q;
        scanf("%d%d", &p, &q);
        s.push_back({p,q});
    }
    sort(s.begin(), s.end());
    total = 0;
}

```



```

pair<int,int> last = s[0];
for (auto seg: s) {
    if (seg.Left > last.Right) { // disjoint
        total += last.Right - last.Left;
        last = seg;
        continue;
    }
    // else part, merge last and s[i]
    last.Right = max(last.Right, seg.Right);
}
total += last.Right - last.Left;
printf("%d\n",total);
return 0;
}

```

下面一個例題非常簡單又直覺，但是很有趣的是，我們稍後會看到他的另外一種形式，當他以另外一個形式出現時，卻似乎比較難理解。

P-4-12. 一次買賣

某商品在某個時期每一天的價格是 $p(1)$, $p(2)$, ..., $p(n)$ 。假設只能先買後賣，請計算買賣一次的最大獲利價差，允許當天買賣，也就是一次都不買 (獲利 0)。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，第二行有 n 個正整數 $p[1]$, $p[2]$, ..., $p[n]$ 。 n 不超過 $1e5$ ，價格皆不超過 $1e9$ 。

輸出：買賣一次的最大價差。

範例輸入：

```

5
3 5 1 4 0

```

範例結果：

```

3

```

範例中暗示你，挑選最大值與最小值並非正確的答案。我們只需要考慮在第 i 天賣出的最大獲利，然後對各個 i 取最大值就是答案了。第 i 天賣出的最大獲利當然就是買在 i 以前出現的最小值，也就是 $i-1$ 的 prefix-minimum。所以我們可以一路掃描過去，維護好 prefix-min，再取較大的價差就好了，以下是範例程式。

```
//P4-12 max price difference, O(n)
#include <bits/stdc++.h>
using namespace std;
#define oo 1000001

int main() {
    int n, max_diff=0; // allow zero
    int p_min=oo; // prefix minimum
    scanf("%d",&n);
    for (int i=0; i<n; i++) {
        int p;
        scanf("%d", &p);
        max_diff=max(max_diff, p - p_min);
        p_min = min(p_min, p);
    }
    printf("%d\n",max_diff);
    return 0;
}
```

下面這一題是教學的經典名題，有人把他在貪心算法，也有人可能歸類到動態規劃。網路上找得到很多解法的介紹。

P-4-13. 最大連續子陣列

輸入一個整數陣列 $A[1:n]$ ，請計算 A 的連續子陣列的最大可能總和，空陣列的和以 0 計算。

Time limit: 1 秒

輸入格式：第一行是正整數 n ，第二行有 n 整數 $A[1], A[2], \dots, A[n]$ 。 n 不超過 $1e5$ ，陣列內容絕對值不超過 $1e9$ 。

輸出：最大可能總和。

範例一輸入： 8 4 12 -17 5 8 -2 7 -3	範例一輸出： 18
範例二輸入： 3 -1 -1 -1	範例二輸出： 0

P-4-13 與 P-4-12 有何關係呢？P-4-12 的輸入是每日價格，如果把每日價格與前一日的價差放在 $A[]$ 中， $A[i]=p(i)-p(i-1)$ ，那麼就變成 P-4-13 了，因為連續一段時間的累積價差就等於最後一日與最前一日的價差

$$A[i]+A[i+1]+\dots+A[j]=(p(i)-p(i-1))+(p(i+1)-p(i))+\dots+(p(j)-p(j-1)) \\ =p(j)-p(i-1)$$

P-4-12 的 $O(n)$ 解法非常直覺明顯，而 P-4-13 的 $O(n)$ 解法則似乎比較不容易。當然你可以把輸入轉換成前一題的輸入，但我們也可以直接看這個題目。最簡單的解法是窮舉 $O(n^2)$ 個所有區間（子陣列），每一個區間再用一個迴圈求和，這樣導致了一個 $O(n^3)$ 的解。改善複雜度降到 $O(n^2)$ 有兩個方法，一種是對所有左端 i ，用一個迴圈求出所有 $[i, j]$ 區間的最大和，這個很容易做到因為

$$\text{sum}(A[i:j+1]) = \text{sum}(A[i:j]) + A[j+1]。$$

另外一種方法是利用前綴和（prefix sum）。先算好前綴和之後，每一個區間和可以用一個減法完成。

但是這兩種都不夠好，從前一題就知道這題可以在 $O(n)$ 求解。令 $f(i)$ 是以 i 為右端（各種可能的左端）的最大區間和。觀察 $f(i+1)$ 與 $f(i)$ 的關係， $f(i+1)$ 所選的左端如果在 i 之前，那麼一定與 $f(i)$ 的左端一樣。所以 $f(i+1)$ 只有以下兩種可能：

- $f(i+1)=f(i)+A[i+1]$ ，左端與 $f(i)$ 相同；或者
- $f(i+1)=A[i+1]$ ，左端在 $i+1$ 。

兩者挑大的就好了，事實上，這就像一代一代累積財產，若 $f(i)>0$ ，爸媽留給你的是正資產，則繼承過來；否則就拋棄繼承。這樣一比喻，這題的解其實也是非常直覺自然的。以下是範例程式。

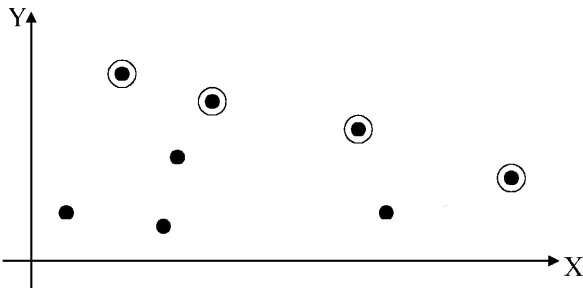
```
//P4-13 max subarray, O(n)
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;
int main() {
    LL n, max_sum=0; // allow zero
    LL p_max=0; // prefix minimum
    scanf("%lld",&n);
    for (int i=0; i<n; i++) {
        LL p;
        scanf("%lld", &p);
        p_max=(p_max>0)? p_max+p : p;
        max_sum = max(max_sum, p_max);
    }
    printf("%lld\n",max_sum);
    return 0;
}
```

```
}
```

下兩題是平面幾何的題目，兩題都常被拿來當作分治演算法 (下一章的主題) 的教材，但事實上都有掃描線的解法。

P-4-14. 控制點 (2D-max)

平面上有 N 個點，第 i 點的座標為 $(x[i], y[i])$ 。若 $x[i] \leq x[j]$ 且 $y[i] \leq y[j]$ ，則我們說第 j 點可以控制第 i 點。我們希望能在輸入的點中，選取最少的點做為控制點，使得每一個輸入的點都至少被一個控制點所控制。輸入 N 個點的座標，請計算出最少要選取多少個控制點。以下圖為例，輸入 8 個點中，右上方圈起的四個點即是最少選取的控制點。



Time limit: 1 秒

輸入格式：每筆測試資料的第一行為一個正整數 N ， $N \leq 1e5$ 。第二行有 N 個非負整數，依序是這些點的 x 座標，第三行有 N 個非負整數，依序是這些點的 y 座標，座標值不超過 $1e9$ 。

輸出：輸出最少的控制點數量。

範例一：輸入 <pre>4 1 2 3 3 1 2 3 3</pre>	範例一：正確輸出 <pre>1</pre>
範例二：輸入 <pre>8 3 1 5 3 4 7 8 9 8 2 5 1 2 4 2 3</pre>	範例二：正確輸出 <pre>4</pre>

這個題目稱為 2D Maximal point，2D 平面上的 maximal point 是指沒有人的 XY 皆大於等於此點，是否會有相同座標點輸入則看題目定義，本題是允許，但本題中如果有兩個相同座標的極大點，只需要選一個。直接的做法可以兩兩比對，複雜度 $O(n^2)$ ，無疑是不夠好。假設我們從左到右掃描這些點，維護好目前找到的解 (maximal points)，假設做到第 $i-1$ 點時的解是 $S(i-1)$ ，考慮當第 i 點 $(x[i], y[i])$ 進入時的狀況。 $S(i-1)$ 中 Y 值不大於 $y[i]$ 的點都應該從解中移除，因為前面點的 x 值都不大於 $x[i]$ ；而 $(x[i], y[i])$ 必然是 $S(i)$ 中的點，因為它具有目前最大的 x 值。要如何刪除 $S(i-1)$ 中 Y 值不大於 $y[i]$ 的點呢？乍看之下以為要使用什麼特別的資料結構，確實，使用 set/map 一定可以做，但其實並不需要。因為 maximal 的點是以 x 值遞增的方式加入，他們的 y 值一定是遞減，只要從後往前刪除就可以了。從後往前刪除，然後自己加入到最後面，所以 Stack 是好的選擇，如果有印象的話，這個方法在前一章找前方的高人那個題目中使用過。請看以下範例程式，為了依照 x 排序，所以程式中定義了一個結構與一個比較函數。因為每個點最多只進入堆疊一次，除了排序外的時間複雜度是 $O(n)$ 。請注意，在排序時，相同 x 值時， y 值小的必須先處理。

```
//P-4-14 2d maximal points (forward sweep + stack)  $O(n \log n)$ 
#include <bits/stdc++.h>
using namespace std;
#define N 100010
#define oo 10000000001
struct Point {
    int x, y;
};
bool less_x(Point a, Point b) {
    return (a.x < b.x) || (a.x == b.x && a.y < b.y);
}
int main() {
    int i, j, n, x;
    scanf("%d", &n);
    Point p[N];
    for (int i=0; i<n; i++)
        scanf("%d", &p[i].x);
    for (int i=0; i<n; i++)
        scanf("%d", &p[i].y);
    sort(p, p+n, less_x); // sort from left to right
    stack<int> S; // keep y-value of current maximal
    S.push(oo); // a dummy maximal (-oo, oo)
    for (i=0; i<n; ++i) { // x from small to large
        while (p[i].y >= S.top())
            S.pop(); // delete smaller y
        S.push(p[i].y);
    }
    printf("%d\n", (int)S.size()-1);
    return 0;
}
```

事實上這個題目有更簡單的寫法。當我們從左往右掃描時，前面找到的 maximal point 有可能需要刪除，如果從右往左掃描呢？因為看到的點的 x 值是遞減，先找到的 maximal point 就一定是解！我們只要記住目前右方的最大 y 值就可以決定下一點是否需要加入解中了（如果他有更大的 y 值）。以下是範例程式，我們用 STL 的 vector 與 pair 來寫，排序時， x 值比較小的排在前面，而相同 x 值的會把 y 比較小的排在前面。除了排序外，時間複雜度顯然是 $O(n)$ ，與前面那支一樣。

```
//P-4-14 2d maximal points backward sweep O(nlogn)
#include<bits/stdc++.h>
using namespace std;
int main() {
    int n, total=0; // num of maximal points
    scanf("%d",&n);
    vector<pair<int,int>> V(n);
    for (int i=0; i<n; i++) {
        scanf("%d", &V[i].first);
    }
    for (int i=0; i<n; i++) {
        scanf("%d", &V[i].second);
    }
    // from x small to large, for same x, y from small to large
    sort(V.begin(), V.end());
    int max_y = -1; // max y value so far
    for (int i=n-1; i>=0; --i) { // backward
        if (V[i].second > max_y) { // maximal point
            total++;
            max_y = V[i].second;
        }
    }
    printf("%d\n",total);
    return 0;
}
```

下一題也是平面上的點的問題，稍難，需要一點幾何特性。

P-4-15. 最靠近的一對(closest pair) (@@)

平面兩點的 L_1 距離為兩點的 x 差值與 y 差值的和，也就說，如果兩點座標是 (a,b) 與 (c,d) ，則 L_1 距離是 $|a-c|+|b-d|$ 。輸入 n 個點的座標，請計算出 L_1 距離最近兩點的 L_1 距離。

Time limit: 1 秒

輸入格式：第一行為一個正整數 n ，接下來 n 行，每行兩個整數 x 與 y 代表一點的座標。 n 不超過 $1e5$ ，座標值絕對值不超過 $1e8$ 。

輸出：最近兩點的 L_1 距離。

範例輸入：

```
4
-1 5
4 0
3 1
-2 -3
```

範例結果：

```
2
```

這個題目也是演算法的經典題，我們在下一章分治演算法也會再談到它，網路上可以找到很多相關資料與教學，大部分是直線距離的版本，也就是兩點之間的距離以直線距離計算，在線性代數中，直線距離稱為 L_2 ，與這一題的 L_1 距離雖然不一樣，但是類似， L_1 距離也有很多重要的應用，例如在 IC 設計上，網路上有很多說明資料。這一題用 L_1 只是為了計算更簡單而且不需要浮點數，演算法其實是一樣的。

想像一根垂直掃描線由右往左掃過去，計算出目前的解，也就是目前已經納入點的最小距離，每次碰到一個新的點，就找出新點與前面點的最近距離，並更新最小距離。如果有多點 x 值若相同，以任意順序並無影響。假設每個新進點都要跟前面的點計算距離，那這個方法跟窮舉沒兩樣，所以我們要想辦法減少計算的點數。

假設目前碰到的點是 $p[i]=(x[i], y[i])$ 而目前已求出來的最小距離是 d ，令 S 是 $p[0] \sim p[i-1]$ 這些點的集合 (如果沒有相同 x 可以簡單說 $p[i]$ 左方的點)，也就是說 S 中任兩點的距離至少是 d 。首先， S 中只有 y 座標值在 $[y[i]-d, y[i]+d]$ 範圍內的點才需要計算，因為其他的點到 $p[i]$ 的距離必然大於 d ；此外，根據相同的理由， S 中 x 座標值小於 $x[i]-d$ 的點都不需要計算，而且不止這回合，以後碰到新點時，這些太左邊的點也都不需要計算了，因為越後面的點 x 座標越大。根據這兩個觀察，我們可以來寫程式了，為了有效地找到 y 座標在 $[y[i]-d, y[i]+d]$ 範圍內的點，我們需要一個動態資料結構，因為需要增加與刪除資料。以下是範例程式，為了排序在第 6 行我們定義了一個 `struct Point` 以及比較函數 `less_x()`。第 19 行將點依照 x 座標由小到大排序，第 20 行宣告一個 `map`，用來存放已經掃過的點，`map` 中以 y 座標當作 `key`，所以會將 y 座標放在第一欄位。第 23 行開始的迴圈是進行逐點掃描的動作，每次碰到一個點，第 25 行以 `lower_bound()` 找到 y 值在 $[y[i]-d, y[i]+d]$ 範圍內的第一點，然後 `while` 迴圈會跑範圍內的所有點，其中如果碰到 x 座標太小的點，就把它移除。第 31 行就更新可能的最小距離。

```
00 // P_4_15 closest pair, sweep line, L1 distance
```

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 typedef long long LL;
04 #define N 100010
05 #define oo 200000001
06 struct Point {
07     int x,y;
08 };
09 bool less_x(Point &s, Point &t) {
10     return s.x < t.x;
11 }
12
13 int main() {
14     int n;
15     scanf("%d",&n);
16     Point p[N];
17     for (int i=0;i<n;i++)
18         scanf("%d%d",&p[i].x,&p[i].y);
19     sort(p, p+n, less_x); // sort by x
20     multimap<int,int> mm; // (y,x), sort by y
21     int min_d = oo; //min distance
22     // for each point from left to right
23     for (int i=0;i<n; i++) {
24         // check [p.y-d, p.y+d]
25         auto it=mm.lower_bound(p[i].y - min_d);
26         while (it!=mm.end() && it->first <= p[i].y+min_d) {
27             if (it->second < p[i].x - min_d) {
28                 it=mm.erase(it); // x too small, out of date
29                 continue;
30             }
31             min_d = min(min_d, \
32                 abs(p[i].x-it->second)+abs(p[i].y-it->first));
33             it++;
34         }
35         mm.insert({p[i].y,p[i].x});
36     }
37     printf("%d\n",min_d);
38     return 0;
39 }
40

```

這個程式因為用了動態資料結構 map，所以有點超過 APCS 的範圍，如果用分治演算法就可以避免 map，我們在下一章會介紹。

且慢，這個題目是不是還沒說明完畢？複雜度呢？複雜度其實是 $O(n \log(n))$ ，雖然不容易從程式碼中看出來，原因是 while 迴圈中，如果不計因為 x 座標太小被刪除的點，每個點計算距離的不會超過 8 個點！而那些因為 x 值被刪掉的反正全部的過程中只會被刪一次。為什麼是 8 點呢？直覺上來看，因為左邊的點之間距離都不小於 d，而我們會納入計算的點都落在寬度為 d 高度 2d 的矩形內，這樣的點數不可能太多，如果是直

線距離，最多 6 點，如果是 L_1 距離，最多是 8 點，這並不特別困難，我們留給讀者自己想一想，網路上也可以找到說明。

4.3. 其他習題

Q-4-16. 賺錢與罰款

泰山派的磨劍坊接了 n 筆訂單，每一筆訂單有需要的工時 $t[i]$ 以及完工要求時間 $d[i]$ ，如果在時間 x 的時後交貨就可以賺 $d[i]-x$ 的錢，也就是說越早完成賺越多，超過時間完成的話，越晚賠越多。磨劍坊每次只能做一件工作，所以要把這 n 筆訂單做一個排程，希望利潤最大，也就是賺最多錢，如果不可能賺錢就要賠最少。泰山派掌門天門道長聽到之後，深怕會賠太多的錢而破產，請你幫忙找出最好的排程。

Time limit: 1 秒

輸入格式：輸入的第一行是工作數 N 。第二行有 N 個正整數，依序是各訂單所需時間 $t[1]$ 、 $t[2]$ 、...、 $t[N]$ 。第三行有 N 個非負整數，依序是各訂單的完工要求 $d[1]$ 、 $d[2]$ 、...、 $d[N]$ ，相鄰以空白間隔。 $N \leq 1e5$ ，時間不超過 1000，完工要求不超過 $1e8$ 。

輸出：最大利益。

範例輸入：

```
5
4 1 5 2 2
2 5 5 3 1
```

範例結果：

```
-16
```

Q-4-17. 死線高手

華山派每個地子都有很多作業，每個作業都有死線 (dead-line)，必須在死線之前完成否則會受到處罰。令狐沖現在有 n 個作業，每個作業需要花的時間是 $t[i]$ 而死線是 $d[i]$ ，此外，每次只能進行一個作業，不可能一次做兩個作業。如果有任何一個作業超過死線，就會被罰到華山之巔面壁一年，請問他是否可能安排作業的順序，讓每個作業的完成時間都不會超過死線，否則小師妹就可能會移情別戀了。

Time limit: 2 秒

輸入格式：輸入包括多筆測資，第一行是測資筆數 T ， $T < 20$ ，以下是 T 筆測資的資料。每筆測資的第一行是作業數 n 。第二行有 n 個正整數，依序是各作業所需時間 $t[1]$ 、 $t[2]$ 、...、 $t[N]$ 。第三行有 n 個正整數，依序是各作業的死線 $d[1]$ 、 $d[2]$ 、...、 $d[N]$ ，相鄰以空白間隔。 $n < 1e5$ ，時間不超過 1000，死線不超過 $1e8$ 。

輸出：依序輸出每筆測資是否所有作業都可以在死線前完成，是則輸出 yes，否則輸出 no。

範例輸入：

```
2
5
2 1 3 1 2
6 6 3 8 9
4
2 1 2 1
3 5 2 6
```

範例結果：

```
yes
no
```

下一題跟 Q-4-8 有關係，是他的變化題。

Q-4-18. 少林寺的櫃姐 (QQ) (*)

少林寺是觀光勝地，每天要接待很多客人，根據數據分析，方丈決定即使在尖峰時期也必須在時間 D 內完成 n 個客人的服務，現在的問題是不知道要開設多少個服務櫃台。假設客人依編號順序到達，第 i 個客人需要 $t(i)$ 的時間來完成他的需求（無論分配到哪一個櫃台），而且每個客人的需求必須在同一個櫃台完成。因為公平的因素，先到客人的服務開始時間不可以大於晚到客人的服務開始時間。因為每開設一個櫃台就要請一位櫃姐，為了節省人事成本，請計算出最少要開設多少櫃台才能讓這 n 個客人的服務都在時間 D 內完成。

Time limit: 1 秒

輸入格式：輸入兩行，第一行是正整數 n 與 D ，第二行是 n 個正整數 $t(1)$ ， $t(2)$ ，...， $t(n)$ ，同行數字間以空白間格。 n 不超過 $1e5$ ， $t(i)$ 不超過 $1e4$ ， D 不超過 $1e9$ 也不小於 $1e4$ ，所以一定有解。

輸出：最少的櫃檯數。

範例輸入：

```
5 9
3 2 5 7 3
```

範例結果：

```
3
```

Q-4-19. 五嶽盟主的會議場所

武林中一共有 n 個門派，每個門派都要上嵩山去見五嶽劍派盟主左冷禪，每個門派的人數以及到達與停留的時間不盡相同，第 i 個門派有 $m(i)$ 個人要去嵩山，到達時間是 $s(i)$ ，而到達後會一直停留到時間 $t(i)$ ，也就是在嵩山的時間是閉區間 $[s(i), t(i)]$ 。左冷禪需要知道最多會有多少人同時在嵩山，以便準備夠大的會議場所，請計算最多在嵩山的人數。

Time limit: 1 秒

輸入格式：第一行是一個正整數 n ，接著的 n 行每一行有三個整數，依序是 $m(i)$ 、 $s(i)$ 與 $t(i)$ ，代表一個門派的人數以及到達與最後停留時間，兩者之間以一個空格區隔。 n 不超過 $1e5$ ，各派人數不超過 $1e4$ ， $0 \leq s(i) < t(i) \leq 1e9$

輸出：依序輸出最多同時在嵩山的人數。

範例輸入：

```
3
5 1 5
2 5 7
4 8 9
```

範例結果：

```
7
```

Q-4-20. 監看華山練功場

華山派有 n 個弟子，每個弟子的練功時間都不盡相同，第 i 個弟子到練功場所練功的時間是區間 $[s(i), t(i)]$ 。最近華山頗不平靜，掌門岳不群要求令狐沖找一些弟子練功時順便監看練功場，對於想要監看的時間區間 $[x, y]$ ，請問他最少只要找幾位弟子，這些弟子的練功時間就可以涵蓋整個 $[x, y]$ 。

Time limit: 1 秒

輸入格式：第一行是個正整數 n ，第二行是兩個整數 x 與 y ，接著的 n 行每一行有兩個整數 $s(i)$ 與 $t(i)$ ，同行相鄰兩數之間空白區隔。 n 不超過 $1e5$ ， $0 \leq x < y \leq 1e9$ ，且對所有 i ， $0 \leq s(i) < t(i) \leq 1e9$ 。

輸出：練功時間可以涵蓋 $[x, y]$ 的最少的弟子數。如果無解輸出 -1 。

範例一：輸入 5 1 10 0 3 1 5 5 7 8 9 6 10	範例一：正確輸出 3
範例二：輸入 5 1 10 0 3 1 5 5 7 8 9 8 10	範例二：正確輸出 -1